Athel — 07 Verification, UAT and Learning

Nov 1, 2026 · @Vaibhav Shukla

After the build, Athel proves the change works in staging using real dashboards and logs, reports the problems it noticed along the way, runs UAT rounds with you, and turns your corrections into rules it applies to every future change.

Purpose, scope and user stories

This module closes the loop between "the code passes its gates" and "the change is right and Athel got better". It covers features F-16 (side findings), F-18 (staging verification), F-19 (UAT rounds) and F-20 (learning).

User stories

ID	As a	I want	So that
US-VER-01	Developer	Athel to tell me how to deploy and what to test, then check staging dashboards, logs and data after I deploy	I know it works in a real environment, not just in tests
US-VER-02	Developer	a clear before-and-after comparison of errors and latency	regressions are obvious
US-FIND-01	Tech lead	existing problems Athel notices to be reported, ranked by risk and effort	we fix easy wins without derailing the change
US-FIND-02	Developer	quick, safe fixes offered as separate small PRs	the main PR stays focused
US-UAT-01	Feature owner	to give feedback in plain words and have Athel sort it into bugs, changes and new scope	I don't have to write tickets
US-UAT-02	Feature owner	each round to show exactly what changed since the last round	I only re-test what moved
US-LEARN-01	Developer	a correction I gave once to apply everywhere after	I never repeat myself

ID	As a	I want	So that
US-LEARN-02	Tech lead	to see, edit and remove what Athel has learned	learning stays under human control

Functional requirements

ID	Requirement	Acceptance criterion
FR-VER-001	Before deploying to staging, record a baseline for every affected service: error rate, p50 and p95 latency, throughput, saturation, open alerts, top errors, over the last 60 minutes.	Baseline file exists before any deploy.
FR-VER-002	Athel never deploys. It reads the GitHub Actions workflows and the Facets setup (through Praxis) to find what a merge or branch push will deploy and where, and writes a plain deploy-and-test guide for the developer.	Audit log has no deploy call; the guide exists before PRs are marked ready.
FR-VER-003	After the developer deploys (detected through read-only tools, or confirmed by the developer), wait for rollout health (all replicas ready, no crash loops), then run the spec's acceptance checks against staging.	Verification report lists each AC with pass or fail and evidence.
FR-VER-004	Compare 30 minutes of post-deploy metrics against baseline using the thresholds in section 4; any breach fails verification.	Fixture with a 3x error spike fails.
FR-VER-005	Check logs for new error signatures that did not exist in the baseline window.	New signature appears in the report with sample (scrubbed).

ID	Requirement	Acceptance criterion
FR-VER-006	A failed verification returns the change to `building` with findings and fix tasks; if the code is already merged, the fix ships as a follow-up PR and the guide tells the developer whether to roll back first.	State transition recorded.
FR-FIND-001	Side findings are recorded with location, evidence, category, severity, effort, fix risk and suggested fix.	Validates against `schemas/finding@1`.
FR-FIND-002	Side findings are never fixed inside a change's tasks.	Diff contains no edits outside planned paths.
FR-FIND-003	At PR time, findings are presented ranked; the human chooses: separate fix PR, ticket, or ignore.	Each finding has a disposition after the PR step.
FR-FIND-004	A chosen quick fix runs as its own mini-change with its own plan, gates and PR.	Fix PR has its own change ID linked to the parent.
FR-UAT-001	UAT feedback is accepted as free text, a list, screenshots or a ticket link, and saved verbatim as `uat/round-N.md`.	Raw input preserved.
FR-UAT-002	Each feedback item is classified: defect (spec met but behaviour wrong), spec change (spec was wrong or incomplete), new scope, or not actionable.	Each item has one class and a reason.
FR-UAT-003	Defects and spec changes produce a new PRD version, a spec diff and a plan diff; new scope is proposed as a separate change.	Plan diff exists; new scope does not enter the current plan silently.
FR-UAT-004	Each round's PR updates list what changed since the previous round per feedback item.	PR comment maps feedback IDs to commits.

ID	Requirement	Acceptance criterion
FR-LEARN-001	After each UAT round and each completed change, candidate lessons are extracted from feedback and repeated review findings.	Candidates file exists with source links.
FR-LEARN-002	A lesson becomes active only after human confirmation, with a scope (repo, service, org) and a check.	Unconfirmed lessons are never applied.
FR-LEARN-003	Active lessons relevant to a task are loaded into spec, developer and reviewer contexts, and violations become review findings.	Fixture lesson is cited in a review finding when violated.
FR-LEARN-004	Lessons can be listed, edited, disabled and deleted by humans; each has a usage and violation count.	`/athel:lessons` shows counts.

Edge cases

Edge case	Required behaviour
Staging is shared and others deploy at the same time	Record other deploys in the window (deploy history); attribute changes only when Athel's deploy is the sole change, otherwise mark the comparison `confounded` and ask
Staging has almost no traffic	Run synthetic checks from the ACs; mark metric comparison `insufficient traffic` rather than passing it
Staging differs from prod (feature flags, data)	Report known differences from config; never claim prod readiness from staging alone
The developer's deploy fails for reasons unrelated to the change	Explain the failure from pipeline or Facets logs and suggest next steps; Athel never retries the deploy itself
UAT feedback contradicts an earlier decision card answer	Surface the conflict with the original decision; ask which wins

Edge case	Required behaviour
UAT feedback is vague ("feels slow")	Ask one clarifying question with measured data offered (current p95 for the screen's calls)
The same correction appears in two changes before being confirmed as a lesson	Raise it as a lesson candidate with both sources and higher priority
A lesson conflicts with a policy or another lesson	Policy wins; conflicting lessons are both marked `conflict` and shown to a human
A lesson is wrong or outdated	Violations that humans repeatedly waive lower its weight; after 3 waivers it is proposed for retirement
Side finding is a security issue	Severity overrides effort: it is always surfaced first, and a ticket is suggested even if ignored

Staging verification

The verifier agent compares the affected services in staging before and after the change, using the same queries both times, and fails the change on any breach of the thresholds below.

Procedure (`/athel:verify`)

1. **Baseline:** for each affected service, run the metric and log queries from the integration playbooks over the last 60 minutes; save to `verify/round-N/baseline.yaml`.
2. **Deploy guide:** write `verify/round-N/deploy-guide.md` in plain words: whether merging auto-deploys and to which environment, the exact steps or commands the developer runs (including any Raptor command, which Athel never runs itself), what to test by hand on staging and later on prod, which dashboards to watch, and how to roll back.
3. **Wait for the developer's deploy:** detect the new version through read-only tools (GitHub Actions runs, Praxis, kubectl), or take the developer's confirmation; record version and time.
4. **Rollout health:** wait up to 10 minutes for ready replicas and no restarts; failure ends verification as `failed: rollout`.
5. **Acceptance checks:** execute each AC as a request or scripted check against staging (generated from the spec's Given/When/Then), plus read-only data checks on the staging database when credentials are configured (doc 04); record request, response

(scrubbed) and verdict. Checks that change data run only if the AC requires it and the plan approved it for staging test accounts.

6. **Observation window:** at least 30 minutes and at least 1,000 requests per affected endpoint, extended up to 4 hours for low-traffic services; if the traffic minimum is still not reached, the comparison is marked `low traffic` and synthetic checks decide.
7. **Compare** with thresholds; write `verify/round-N.md` with a verdict per service and a short list of what the developer should still check on prod after release.
8. **Contract cleanup check** (for deferred removal tasks): measure traffic on old fields or endpoints; removal is proposed only when it is zero for 7 days in every environment.

Thresholds (defaults in `policies/defaults/verify.yaml`)

Signal	Fails when
Error rate	Above baseline by more than 0.25 percentage points, or more than 1.5x baseline when baseline is above 0.25%
Latency	p95 more than 10% above baseline, p99 more than 20% above baseline, or above the `risk_notes` budget
New error signatures	Any new signature at all, with samples (scrubbed) in the report
Restarts	Any crash loop or OOM kill
Consumer health	Any consumer in the blast radius shows a new error signature or lag growth above 1.5x
Acceptance checks	Any AC fails
Data checks	Any staging data check fails (when configured)

The verifier writes facts only as `volatile` (doc 03); verification results never become long-lived knowledge.

Side findings

Any agent can record a side finding at any time; findings are presented once, at PR time, ranked so the cheap and safe ones are easy to accept.

Finding record (`findings.yaml`)

```
- id: SF-3
  found_by: reviewer
  found_in: {step: build, task: T-03}
  location: user-service/src/profile/address.ts:88
  category: bug            # bug | security | performance | reliability |
dead-code | dependency | test-gap | docs
  severity: medium         # low | medium | high | critical
  effort: S                # S < 30 lines, M < 150, L larger
  fix_risk: low            # how likely the fix itself breaks something
  evidence: "Postcode trimmed before validation only on create, not update;
212 records with trailing spaces (scout F-0233)."
  suggested_fix: "Apply the same trim in update path; add test."
  disposition: null        # fix_pr | ticket | ignore (set by human)
```

Ranking: security findings first; then by `severity` high to low; within a severity, `effort` small and `fix_risk` low first. Findings with `effort: S` and `fix_risk: low` are labelled **quick win**.

Presentation at PR time: "Found 4 issues outside this change. 2 quick wins: SF-3, SF-5. Fix them as separate PRs, file tickets, or ignore?" One answer per finding, or one answer for all.

Fix PR flow: a chosen finding becomes a child change (`CHG-042-F3`) with a one-task plan, the same gates and review loop, its own branch (`side_finding_branch` in `git.yaml`), and a PR linked to the parent. It goes through blast radius too, so even a "quick" fix cannot break consumers unnoticed.

Tickets: with a ticketing tool connected, Athel drafts the ticket (title, evidence, suggested fix) and shows it for approval before creating it (tier W2).

UAT rounds

A UAT round turns your plain-language feedback into a precise, reviewable delta: a new PRD version, a spec diff and a plan diff, so round 2 changes only what you asked to change.

Procedure (`/athel:uat`)

1. Save your input verbatim to `uat/round-N.md` ; split it into items `U-N.1` , `U-N.2` , …
2. Classify each item (FR-UAT-002) with a one-line reason, and show the classification back for a quick confirm.
3. **Defects:** add a failing test that reproduces the defect first, then a fix task.
4. **Spec changes:** write `prd/vN+1.md` with tracked edits; the architect produces the spec diff; the planner produces a plan diff. Plan diff needs approval (doc 05).

5. **New scope:** propose a new change with its own ID; it never joins the current plan without approval of a scope expansion.
6. Run the build loop on the new tasks only; re-run full gates and verification for the affected services.
7. Post a round summary on each PR: feedback item → commits → tests → verification result.
8. Extract lesson candidates (section 7).

Round summary (example)

Item	Feedback	Class	Result
U-2.1	Success message should show the masked number	Spec change	REQ-6 added; T-11 done; AC-9 passes in staging
U-2.2	Error shows for numbers with spaces	Defect	Repro test added; fixed in T-12
U-2.3	Also let users verify via OTP	New scope	Proposed as CHG-051

Exit: the human accepts the round (`done`) or starts another. After 5 rounds on one change, Athel suggests splitting remaining items into a follow-up change to keep PRs reviewable.

Learning

Athel learns by turning confirmed corrections into small, scoped, checkable rules, not by changing the model; every lesson is a file a human can read, edit or delete.

Where lessons come from

Source	Candidate when
UAT spec changes and defects	The correction is general ("always…", "never…"), not specific to this feature
Review findings	The same rule ID is violated in 3 or more tasks
Decision card answers	The same question type gets the same answer twice
Waivers	A gate or rule is waived repeatedly (candidate to adjust policy instead)

Source	Candidate when
Incidents or escaped defects	A human links a post-merge problem to a change

Lesson record (`lessons/L-0007.yaml`)

```
id: L-0007
status: active            # candidate | active | conflict | retired
scope: {org: acme, services: ["*"], paths: ["**/ui/**",
"**/notifications/**"]}
rule: Mask phone numbers in any user-facing text (show last 4 digits).
why: UAT U-2.1 on CHG-042; privacy guideline.
applies_to: [spec, developer, reviewer]
check:
  kind: review_rule       # review_rule | lint | test_template | gate
  rule_id: privacy.mask-phone
sources: [CHG-042/uat/round-2.md#U-2.1]
confirmed_by: me@example.com
stats: {applied: 14, violations_caught: 3, waived: 0}
```

Lifecycle

1. **Extract:** after each UAT round and change completion, the architect agent proposes candidates with sources.
2. **Confirm:** the human accepts, edits or rejects each candidate; only accepted ones become `active`.
3. **Apply:** at each task, the helper CLI selects lessons whose scope matches the repo and paths (deterministic match, no model judgement) and injects them into the spec, developer and reviewer contexts; reviewers cite violated lessons by ID.
4. **Strengthen:** a lesson with a mechanical check (lint rule, test template, gate) is preferred; the architect proposes turning frequent lessons into such checks.
5. **Retire:** after 3 human waivers, or no matches in 180 days, the lesson is proposed for retirement; conflicts are resolved by a human.

Limits: at most 15 lessons are injected per task, ranked by scope specificity and recent violation count, to keep context small. All lessons stay visible in `/athel:lessons` regardless.